DEEP LEARNING WITH KERAS AND TENSORFLOW

Viva Questions and Answers - Short and Important

Unit	Topic	Questions
1	Neural Networks Foundations and Keras/TensorFlow	29
2	Convolutional Neural Networks	22
3	Sequence Models - RNN, LSTM, GRU and Transformers	20
4	Model Evaluation, Debugging and Data Pipelines	20
5	Autoencoders, GANs and Representation Learning	20

Total: 111 questions, plus a formula quick-sheet at the end.

Unit 1: Neural Networks Foundations and Keras/TensorFlow (Weightage 20)

Q1. What is deep learning?

A part of machine learning that uses neural networks with many layers to learn features from data automatically.

Q2. Difference between machine learning and deep learning?

In ML the features are chosen by hand; in DL the network learns the features itself. DL needs more data and more computing power.

Q3. What is a neuron?

The basic unit of a network. It finds the weighted sum of its inputs plus a bias and passes it through an activation function: **y = f(w.x + b)**.

Q4. What is a perceptron?

The simplest neural network: one neuron with a step activation. It can solve only linearly separable problems (it cannot solve XOR).

Q5. What is an MLP?

Multi-Layer Perceptron: an input layer, one or more hidden layers and an output layer, all fully connected, with non-linear activations. It can solve XOR.

Q6. Biological neuron vs artificial neuron?

Dendrites = inputs, synapse strength = weights, cell body = sum + activation, axon = output.

Q7. What are tensors?

N-dimensional arrays: scalar (0-D), vector (1-D), matrix (2-D), and higher. All data and weights in TensorFlow are tensors.

Q8. Why is linear algebra and probability needed?

Data and weights are vectors and matrices (forward pass = matrix multiplication). Probability is used in softmax outputs, cross-entropy loss and in dealing with uncertainty.

Q9. What is a deep feedforward network?

A network in which information flows in one direction only, from input to output, with no loops. It has several hidden layers.

Q10. Why do we need activation functions?

They add non-linearity. Without them, many layers would collapse into one linear function and could not learn complex patterns.

Q11. Write the common activation functions.

Sigmoid (0 to 1, binary output, vanishing gradient), **Tanh** (-1 to 1, zero-centred), **ReLU** = max(0, x) (fast, most used in hidden layers), **Softmax** (probabilities of many classes that add to 1).

Q12. What is the dying ReLU problem?

If a ReLU neuron always gets negative input, its output and gradient are 0 and it stops learning. Leaky ReLU is a fix.

Q13. What is backpropagation?

The method to find the gradient of the loss for every weight using the chain rule, going from the output layer back to the input layer. The weights are then updated using gradient descent.

Q14. Forward pass vs backward pass?

Forward pass: input goes through the layers to give a prediction and the loss. Backward pass: gradients are calculated and weights are updated.

Q15. What is a loss function? Name some.

It measures the error between the prediction and the true value. **MSE** for regression, **binary cross-entropy** for two classes, **categorical cross-entropy** for many classes.

Q16. What is gradient descent?

An optimization method that updates the weights in the direction that reduces the loss: **w = w - learning_rate x dL/dw**.

Q17. Batch, stochastic and mini-batch gradient descent?

Batch uses all data per update; stochastic (SGD) uses one sample; mini-batch uses a small group (e.g. 32). Mini-batch is the most used.

Q18. What is the learning rate?

The step size of each weight update. Too high: the loss jumps and may diverge. Too low: training is very slow.

Q19. What is the Adam optimizer?

Adaptive Moment Estimation. It combines momentum and RMSprop, giving each weight its own adaptive learning rate. It is fast and a good default.

Q20. Epoch, batch size and iteration?

Epoch = one full pass over the training data. Batch size = samples per update. Iteration = one update (iterations per epoch = data size / batch size).

Q21. What is the vanishing gradient problem?

In deep networks the gradients become very small in the early layers, so those layers learn very slowly. Fixes: ReLU, proper weight initialization, batch norm, residual connections.

Q22. What is overfitting? How to reduce it?

The model does well on training data but badly on new data. Reduce it with more data, data augmentation, regularization (L1/L2), dropout, early stopping and a simpler model.

Q23. What is underfitting?

The model is too simple and does badly on both training and test data. Fix with a bigger model, more features and longer training.

Q24. What is regularization? L1 vs L2?

A way to stop overfitting by penalizing large weights. L1 adds the sum of absolute weights (gives sparse weights); L2 adds the sum of squared weights (weight decay, small weights).

Q25. What are learning curves?

Plots of training and validation loss/accuracy against epochs. A big gap between them means overfitting; both high loss means underfitting.

Q26. What are Keras callbacks? Name a few.

Functions called during training. **EarlyStopping**, **ModelCheckpoint**, **ReduceLROnPlateau**,

TensorBoard.

Q27. What is TensorBoard?

TensorFlow's visualization tool to see loss and accuracy curves, the model graph and weight histograms.

Q28. Sequential API vs Functional API?

Sequential: a simple stack of layers, one input and one output. Functional: flexible graphs with many inputs/outputs, branches and skip connections.

Q29. What do compile(), fit(), evaluate() and predict() do?

compile sets the optimizer, loss and metrics; fit trains the model; evaluate gives loss and metrics on test data; predict gives outputs for new data.

Unit 2: Convolutional Neural Networks (Weightage 20)

Q1. What is a CNN?

A neural network made for grid-like data such as images. It uses convolution layers to find local features (edges, shapes) automatically.

Q2. What is the convolution operation?

A small filter (kernel) slides over the image; at each position the dot product is taken, giving a **feature map**.

Q3. What is pooling? Types?

Down-sampling of the feature map to reduce size and computation. **Max pooling** takes the maximum of each window; **average pooling** takes the average. It gives small translation invariance.

Q4. What is padding? Same vs valid?

Adding border pixels (usually zeros) around the image. **Valid** = no padding (output gets smaller); **same** = padding so output size equals input size.

Q5. What is stride?

The number of pixels the filter moves at each step. A larger stride gives a smaller output.

Q6. Output size formula of a convolution?

(N - F + 2P) / S + 1, where N = input size, F = filter size, P = padding, S = stride.

Q7. What is parameter sharing?

The same filter weights are used at every position of the image. This greatly reduces the number of parameters compared with a fully connected layer.

Q8. What is equivariance?

If the input shifts, the feature map shifts by the same amount (translation equivariance of convolution). Pooling adds a little invariance.

Q9. Why CNN and not MLP for images?

An MLP needs a huge number of weights and ignores the 2-D structure. A CNN uses local connections and shared weights, so it has fewer parameters and detects features anywhere in the image.

Q10. What is a Flatten layer?

It converts the 2-D/3-D feature maps into a 1-D vector so that Dense layers can use them.

Q11. Typical CNN layer order in Keras?

Conv2D -> MaxPooling2D (repeated) -> Flatten -> Dense -> Dense with softmax.

Q12. What is LeNet-5?

An early CNN (Yann LeCun, 1998) for handwritten digits. It has 7 layers: convolution and average pooling layers, then fully connected layers; input 32 x 32.

Q13. What is AlexNet?

Winner of ImageNet 2012. 8 layers, used **ReLU**, **dropout**, data augmentation and GPU training. It started the deep learning boom.

Q14. What is VGG?

A deep CNN (16 or 19 layers) that uses only small 3 x 3 filters stacked repeatedly, giving a simple and uniform design.

Q15. What is ResNet?

A very deep CNN with **skip (residual) connections**: output = F(x) + x. They solve the vanishing gradient and degradation problems and allow 100+ layers.

Q16. What is YOLO?

You Only Look Once: a real-time object detector that predicts bounding boxes and class probabilities for the whole image in one pass.

Q17. What is transfer learning?

Using a model pre-trained on a large dataset (e.g. ImageNet) for a new task. Freeze the base layers, train a new top layer; then optionally **fine-tune** some layers. Needs less data and time.

Q18. What is dropout?

Randomly switching off a fraction of neurons (e.g. 50%) during training to stop co-adaptation and overfitting. It is not applied at test time.

Q19. What is batch normalization?

Normalizing the inputs of a layer for each mini-batch (mean 0, variance 1). It gives faster, more stable training and a small regularizing effect.

Q20. What is data augmentation?

Creating new training images by flipping, rotating, zooming, cropping or changing brightness. It increases data variety and reduces overfitting.

Q21. Image classification vs object detection vs segmentation?

Classification: one label for the whole image. Detection: what and where (bounding boxes). Segmentation: a class for every pixel.

Q22. What are the applications of CNN?

Image recognition, object detection, image segmentation, face recognition, medical imaging and self-driving cars.

Unit 3: Sequence Models - RNN, LSTM, GRU and Transformers (Weightage 20)

Q1. What is an RNN?

A network for sequential data. It has a hidden state that works as memory, and the same weights are used at every time step.

Q2. What is unfolding (unrolling) of an RNN?

Drawing the recurrent loop as a chain of copies of the network, one for each time step, with shared weights.

Q3. What is backpropagation through time (BPTT)?

Backpropagation applied on the unrolled RNN; gradients flow backward through all the time steps.

Q4. What is the long-term dependency problem?

In long sequences the gradients vanish (or explode) over many time steps, so a simple RNN cannot remember far-away information. Solutions: LSTM, GRU, skip connections, gradient clipping.

Q5. What is an LSTM?

Long Short-Term Memory: a special RNN with a **cell state** and three gates that control what to forget, store and output. It learns long-term dependencies.

Q6. Name the three gates of an LSTM.

Forget gate (what to discard), **input gate** (what new information to store) and **output gate** (what to output from the cell).

Q7. What is a GRU?

Gated Recurrent Unit: a simpler LSTM with two gates (**reset** and **update**) and no separate cell state. It has

fewer parameters and trains faster.

Q8. LSTM vs GRU?

LSTM: 3 gates + cell state, more powerful, slower. GRU: 2 gates, faster, similar accuracy on many tasks.

Q9. What are stacked RNNs?

Several recurrent layers on top of each other. The lower layers must return the full sequence (return_sequences=True) for the next layer.

Q10. What is a bidirectional RNN?

It reads the sequence forward and backward and joins the outputs, so each step sees both past and future context. Not suitable where the future is unknown (real-time forecasting).

Q11. What is the attention mechanism?

It lets the model focus on the most relevant parts of the input at each step by taking a weighted sum of the input states. It solves the fixed-vector bottleneck of encoder-decoder RNNs.

Q12. What is self-attention?

Each element of a sequence attends to all other elements of the same sequence, using Query, Key and Value vectors, to learn the relation between them.

Q13. What is a Transformer?

A model based only on self-attention (no recurrence) with an encoder-decoder structure ("Attention Is All You Need", 2017). It processes all positions in parallel and handles long dependencies well.

Q14. Why are Transformers better than RNNs?

Parallel training (faster), better long-range dependencies, no vanishing gradient over time. Positional encoding gives the order of the words.

Q15. What is the encoder-decoder architecture?

The encoder converts the input sequence into a representation; the decoder produces the output sequence from it. Used in translation, summarization and captioning.

Q16. Types of sequence problems?

One-to-many (image captioning), many-to-one (sentiment analysis), many-to-many (translation, time-series forecasting).

Q17. What is an Embedding layer?

It maps each word index to a dense, learnable vector so that similar words have similar vectors.

Q18. Applications of sequence models?

Time-series forecasting, text generation, machine translation, sentiment analysis, speech recognition and NLP tasks.

Q19. Which Keras layers are used for sequence models?

SimpleRNN, LSTM, GRU, Bidirectional, Embedding, and Dense for the output.

Q20. Why is data scaled and windowed in time-series forecasting?

Scaling (0 to 1) helps training; a sliding window of past values is used as the input (X) and the next value as the target (Y).

Unit 4: Model Evaluation, Debugging and Data Pipelines (Weightage 20)

Q1. Define accuracy, precision, recall and F1.

Accuracy = (TP+TN)/total. Precision = TP/(TP+FP). Recall = TP/(TP+FN). F1 = 2 x Precision x Recall / (Precision + Recall).

Q2. What is a confusion matrix?

A table of actual against predicted classes showing TP, FP, TN and FN. All other metrics are calculated from it.

Q3. What do TP, FP, TN and FN mean?

True Positive: positive predicted correctly. False Positive: wrongly predicted positive. True Negative:

negative predicted correctly. False Negative: positive missed.

Q4. When is accuracy misleading?

With imbalanced data. If 95% of the samples are of one class, predicting that class always gives 95% accuracy but is useless. Use precision, recall and F1.

Q5. Precision vs recall - which is important when?

Recall is important when missing a positive is costly (disease detection). Precision is important when a false alarm is costly (spam filter).

Q6. What is the ROC curve and AUC?

ROC plots True Positive Rate against False Positive Rate at different thresholds. AUC is the area under it: 1 = perfect, 0.5 = random guessing.

Q7. What is a baseline model?

A simple model (e.g. predicting the majority class or the mean) used as a reference. A deep model must beat it to be worth using.

Q8. Parameters vs hyperparameters?

Parameters (weights, biases) are learned during training. Hyperparameters (learning rate, batch size, layers, epochs) are set by us before training.

Q9. What is hyperparameter tuning? Methods?

Finding the best hyperparameter values. **Manual** search, **grid search** (tries all combinations) and **random search** (tries random combinations; usually more efficient than grid).

Q10. Grid search vs random search?

Grid search is exhaustive and expensive when there are many hyperparameters. Random search covers the space faster and often finds a good setting with fewer trials.

Q11. What is early stopping?

Stopping training when the validation loss stops improving for some epochs (patience), and keeping the best weights. It prevents overfitting.

Q12. Which callbacks are used for training control?

EarlyStopping, ModelCheckpoint (save best model), ReduceLROnPlateau (lower the learning rate when stuck) and TensorBoard.

Q13. How do you debug a deep learning model?

Check the data and labels, start with a small model, try to overfit a small batch, check the learning rate and loss, view curves in TensorBoard and do error analysis.

Q14. What is error analysis?

Looking at the examples that the model gets wrong to find patterns (bad labels, a class that is confused, poor image quality) and then fixing the data or the model.

Q15. Train, validation and test sets?

Train: to learn weights. Validation: to tune hyperparameters and for early stopping. Test: only once at the end for the final unbiased performance.

Q16. What is data leakage?

When information from the test/validation data gets into the training (e.g. scaling on the full data). It gives falsely good results.

Q17. What is the tf.data API?

TensorFlow's API to build fast input pipelines: from_tensor_slices -> map (preprocess) -> shuffle -> batch -> cache -> prefetch.

Q18. Why use tf.data and prefetch?

It loads and preprocesses data in parallel with training, so the GPU is not left waiting, and it can handle data too large for memory.

Q19. Common preprocessing techniques?

Normalization/scaling, resizing images, one-hot encoding of labels, tokenization and padding of text, handling missing values and augmentation.

Q20. Metrics for regression?

MSE, RMSE, MAE and R-squared.

Unit 5: Autoencoders, GANs and Representation Learning (Weightage 20)

Q1. What is an autoencoder?

An unsupervised neural network that compresses the input into a small **latent code** (encoder) and then rebuilds the input from it (decoder). The loss is the reconstruction error (MSE).

Q2. Parts of an autoencoder?

Encoder, bottleneck (latent space) and decoder.

Q3. What is an undercomplete autoencoder?

The bottleneck has fewer neurons than the input, so it must learn only the most important features. Used for dimensionality reduction.

Q4. What is a sparse autoencoder?

A penalty (e.g. L1) on the hidden activations makes only a few neurons active at a time. The hidden layer can even be larger than the input.

Q5. What is a denoising autoencoder?

The input is corrupted with noise and the network is trained to rebuild the clean original. It learns robust features and is used to remove noise.

Q6. What is a VAE?

Variational Autoencoder: the encoder gives a **mean and a variance** (a probability distribution) instead of one point. A sample z is taken from it and decoded. It can generate new data.

Q7. What is the loss of a VAE?

Reconstruction loss + KL divergence. The KL term pushes the latent distribution close to a standard normal distribution.

Q8. What is the reparameterization trick?

z = mean + sigma x epsilon, with epsilon from N(0, 1). It makes the random sampling differentiable so that backpropagation can work.

Q9. Autoencoder vs VAE?

An autoencoder maps an input to a fixed point and is mainly for compression/denoising. A VAE maps it to a distribution with a smooth latent space and can generate new samples.

Q10. What is a GAN?

Generative Adversarial Network (Ian Goodfellow, 2014): two networks compete. The **generator** makes fake data from noise; the **discriminator** tells real from fake. Both improve together.

Q11. Explain the GAN training idea.

The discriminator is trained to classify real and fake correctly; the generator is trained to fool the discriminator. At the end, the generator makes realistic data.

Q12. Problems in training GANs?

Mode collapse (the generator gives only a few kinds of outputs), unstable training and vanishing gradients.

Q13. GAN vs VAE?

GANs give sharper, more realistic images but are harder to train. VAEs are stable and have a useful latent space, but their images are often blurrier.

Q14. Applications of autoencoders?

Dimensionality reduction, image denoising, anomaly detection, data compression and pre-training.

Q15. Applications of GANs?

Image generation, super-resolution, image-to-image translation, style transfer, data augmentation and deepfakes.

Q16. What is a generative model vs a discriminative model?

A generative model learns the data distribution and can create new data (VAE, GAN). A discriminative model learns the boundary between classes (a classifier).

Q17. What is unsupervised pre-training?

Training a network first on a lot of unlabeled data (e.g. as an autoencoder) and then fine-tuning it on a small labeled dataset.

Q18. What is representation learning?

Learning useful features (representations) of the data automatically, which can then be used for many tasks.

Q19. What is neural style transfer?

Combining the content of one image with the style (texture, colour) of another image using a CNN.

Q20. What is the latent space?

The low-dimensional space of the bottleneck where the compressed features of the data are stored.

Formula Quick-Sheet

Item	Formula
Neuron	y = f(w.x + b)
ReLU / Sigmoid	max(0, x) / 1 / (1 + e^-x)
Gradient descent	w = w - lr x dL/dw
MSE	(1/n) x sum of (y - y_pred)^2
Conv output size	(N - F + 2P) / S + 1
Precision / Recall	TP / (TP + FP) / TP / (TP + FN)
F1 score	2PR / (P + R)
Accuracy	(TP + TN) / (TP + TN + FP + FN)
Residual block	output = F(x) + x
VAE loss	reconstruction loss + KL divergence
Iterations per epoch	data size / batch size